MIT 8103 Advanced Database Systems Portfolio — AquaSense+

Student: Anoliefo Kenechukwu · **Matric no.:** 301910291.

GitHub repository:
github.com/Techkene/Anoliefo_Kenechukwu_301910291_Advanced_Database_Portfolio

1. Selected organisation: AquaSense+

All five portfolios use one case study: **AquaSense+** (www.aquasenseapp.com), a record-keeping platform for Nigerian fish farmers, traders and cooperatives. Farmers stock ponds, record daily feeding, water quality and deaths, then harvest and sell; the app shows feed use, survival and profit per production cycle. Its users are farmers, farm staff, traders, cooperative administrators and extension officers. The synthetic but realistic dataset holds 180 farms in 16 states, 905 ponds, 3,764 production cycles, about 506,000 feeding records and 1.03 million water-quality readings. Everything was run on my own Windows laptop with PostgreSQL 16 and Jupyter Notebook.

2. Summary of the five portfolios

Portfolio	What I built	Key evidence
1. Database design and modelling	18-table PostgreSQL schema, ER diagram, keys and constraints, normalisation from UNF to BCNF	All 7 deliberate rule violations rejected (negative feed stock, two active batches in one pond, invalid pH and others)
2. Query processing and optimisation	8 real queries with `EXPLAIN (ANALYZE, BUFFERS)` before and after composite, partial and covering indexes; a materialized view	Batch report 2,274 ms → 3 ms; pond dashboard 208 ms → 0.9 ms; rewriting a date filter as a range cut pages read from 59 to 4. Indexes made inserts about 24% slower
3. Transactions and concurrency	3 transactional procedures; COMMIT, ROLLBACK and SAVEPOINT; two live sessions racing each other	Lost update (80 kg instead of 50) fixed by `FOR UPDATE`; write skew (1,200 kg sold of 1,000) stopped by SERIALIZABLE; deadlock fixed by lock ordering
4. NoSQL and advanced data models	Redis key-value layer on [Memurai / Redis on WSL]: hashes, sorted sets, stream with consumer groups, geo, TTL, Lua; MongoDB document design	Sensor stream raised 4 low-oxygen alerts and wrote back to PostgreSQL; duplicate uploads blocked by `SET NX`. With good indexes PostgreSQL matched Redis on single lookups
5. Distributed and cloud	4-server lab copied from my database: streaming replica, sync vs async replication, `hash(farm_id)` sharding, failover; AWS design	Sync replication blocked writes when the replica was down (CAP); 1,033,625 readings sharded 56/44; failover to timeline 2 with 0 rows lost

The proposed cloud design places the sync API in the AWS Local Zone in Lagos and runs RDS for PostgreSQL Multi-AZ, a read replica and ElastiCache in af-south-1, with point-in-time recovery. Money and stock choose consistency, reports choose latency, and alerts choose availability.

3. Individual technical reflection

I chose AquaSense+ because I know how fish farmers keep records, so every table maps to something a farmer actually does. The hardest design decision was storing the feed-stock balance instead of deriving it from hundreds of thousands of rows; I protected it with transactions and a `CHECK` constraint, which linked Portfolio 1 to Portfolio 3.

Running everything on Windows taught me as much as the theory. I fixed a missing script and a password-authentication failure, moved my password out of the notebooks into a prompt, and learned that Git keeps tracking `.ipynb_checkpoints` until it is removed with `git rm --cached`. The Redis installer failed several times before I had a working server, and for Portfolio 5 I built a separate four-server copy rather than crash my main PostgreSQL service.

My main lessons: indexing mattered more than changing technology, since PostgreSQL matched Redis once the right index existed; my laptop was slower than the original test machine, yet every plan read exactly the same pages, which showed why plans matter more than raw timings; and watching a commit wait on `SyncRep` made the CAP theorem concrete. Next, I would automate failover with Patroni and partition the sensor table by time.

4. Screenshots and Supporting Evidence

Each repository folder holds the scripts, a notebook with saved outputs, an `evidence/` folder and a `screenshots/` folder.
Screenshots documenting the implementation, execution and results of the various tasks are provided in the Appendices for reference:
Figure 1: Entity Relationship (ER) Diagram

Figure 2: Query Execution Plan (Q4) Before and After Indexing.

Figure 3: Lost Update and Deadlock Demonstrations.

Figure 4: Redis Streams and Alert Notifications.

Figure 5: Synchronous Replication (SyncRep) Blocking, Sharded Query Plan and Failover.

Figure 6: Cloud Architecture.

5. AI use declaration

I used Claude (Anthropic) to generate first versions of the SQL, Python scripts and notebooks, to adapt them for Windows, and to draft the READMEs and this summary. I ran every portfolio myself on my own laptop; all evidence, screenshots and figures come from my runs, and I diagnosed and fixed the problems that arose.

6. References

- PostgreSQL Global Development Group (2026) *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/
- Redis Ltd. (2026) *Redis documentation*. https://redis.io/docs/
- MongoDB Inc. (2026) *MongoDB Manual*. https://www.mongodb.com/docs/manual/
- Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Pearson.
- Kleppmann, M. (2017) *Designing Data-Intensive Applications*. O'Reilly.

7. Appendixes

Figure 1: Entity Relationship (ER) Diagram

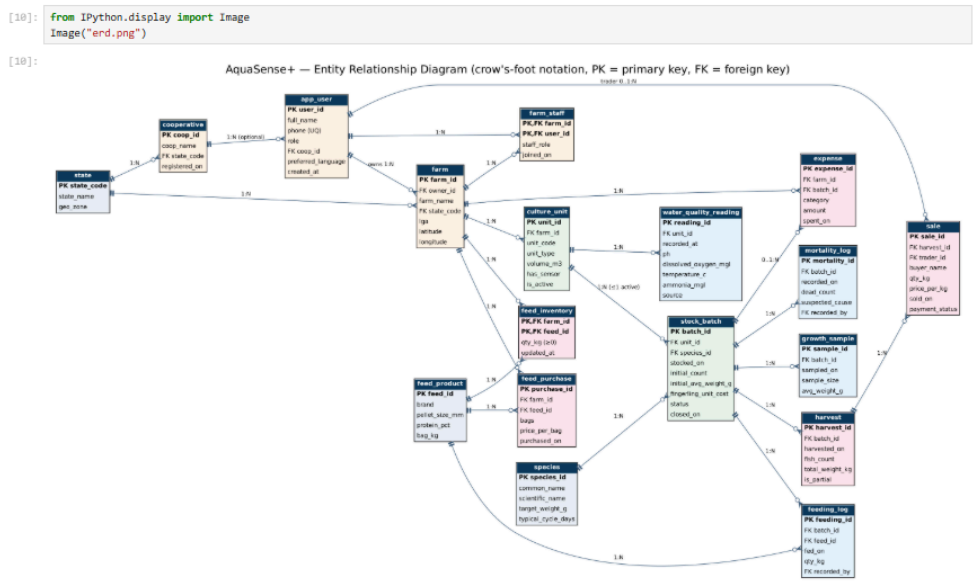


Figure 2: Query Execution Plan (Q4) Before and After Indexing.

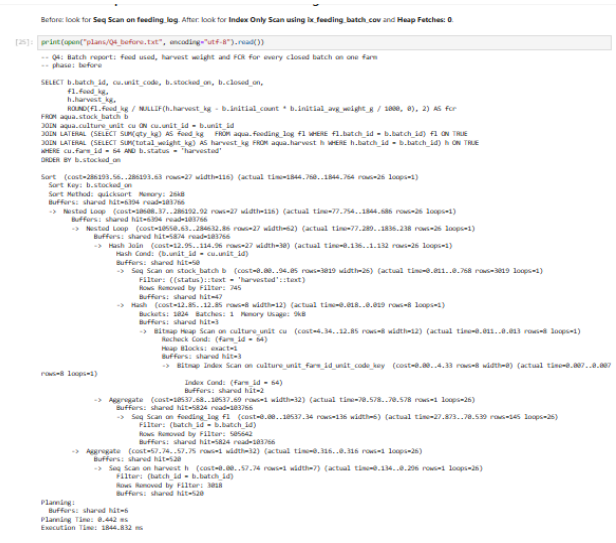


Figure 3: Lost Update and Deadlock Demonstrations.

Every `CHECK` line is the outcome of one demo; `waited` lines are lock waits; `ERROR` lines are the database stopping an anomaly.

```
]: for l in run.stdout.splitlines():
       if any(k in l for k in ("CHECK", "waited", "ERROR")):
           print(l[:170])
```

```
[  0.37s] CHECK   final stock = 80.00 kg  (expected 50) -> A's 30 kg deduction was LOST
[  1.80s] B       (waited 1.19s for A's row lock)
[  1.91s] CHECK   final stock = 50.00 kg  (expected 50) -> correct
[  2.15s] B       ERROR 40001 ERROR:  could not serialize access due to concurrent update
[  2.30s] CHECK   final stock = 50.00 kg  (expected 50) -> correct after one retry
[  2.60s] CHECK   A read 100.00 then 250.00 -> CHANGED inside one transaction (non-repeatable read)
[  2.87s] CHECK   A read 100.00 then 100.00 -> STABLE (same snapshot for the whole transaction)
[  3.43s] CHECK   total sold = 1200.00 kg of 1000 kg -> OVERSOLD (anomaly allowed)
[  3.75s] B       ERROR 40001 ERROR:  could not serialize access due to read/write dependencies among transactions
[  3.85s] CHECK   total sold = 600.00 kg of 1000 kg -> rule held
[  5.18s] B       ERROR AQ003 ERROR:  Oversell blocked: 1000.00 kg harvested, 600.00 kg already sold, 600 kg requested
[  5.32s] CHECK   total sold = 600.00 kg of 1000 kg -> rule held
[  6.72s] A       ERROR 40P01 ERROR: deadlock detected DETAIL: Process 10212 waits for ShareLock on transaction 977; blocked by process 9872. Process 98
72 waits for Share
[  6.73s] CHECK   outcome: {'A': 'aborted (deadlock victim)', 'B': 'committed'} -> PostgreSQL detected the cycle after deadlock_timeout (1 s) and aborte
d one
[  7.91s] B       (waited 0.80s — queued behind A instead of deadlocking)
[  8.02s] CHECK   balances [(1, Decimal('120.00')), (5, Decimal('80.00'))]; total = 200.00 kg (expected 200, farm1 = 120, farm5 = 80)
```

Figure 4: Redis Streams and Alert Notifications.

```
[5]: keep = ("Connected", "Loaded", "Single client", "screens/s", "old readings", "raised", "XPENDING", "writer-1",
             "call 1", "call 2", "used_memory", "aof_enabled", "farms\n", "[1,", "[0,")
     lines = run.stdout.splitlines()
     for i, l in enumerate(lines):
         if any(k in l for k in keep) or l.startswith("redis> SET sync") or l.startswith("redis> GET otp") or l.startswith("redis> GEOSEARCH"):
             nxt = lines[i + 1] if i + 1 < len(lines) and l.startswith("redis>") else ""
             print(l[:160]);
             if nxt: print(nxt[:160])
```

```
Connected to 7.2.4 (Memurai) on 127.0.0.1:6379
Loaded 905 latest-reading hashes, 23,584 time-window entries, 180 geo-indexed farms in 4.83s. DBSIZE = 2171
Single client, median of 2,000 calls: Redis HGETALL = 404 µs | PostgreSQL (indexed) = 354 µs.
32 concurrent Python clients x 300 farm-overview screens: Redis = 857 screens/s | PostgreSQL = 1,135 screens/s.
       48 old readings removed, 36 kept
worker-1 processed 12 messages, raised 4 low-oxygen alerts
redis> XPENDING stream:readings alert-workers
       {'pending': 0, 'min': None, 'max': None, 'consumers': []}
writer-1 batch-inserted 12 rows into PostgreSQL -> verified in SQL: (12, Decimal('1.70')) (count, min DO)
redis> SET sync:device-7f3a:rec-000184 done NX EX 86400
       True
redis> SET sync:device-7f3a:rec-000184 done NX EX 86400   (retry of the same record)
       None
redis> GET otp:+2348030015838   (after 2.2 s, code was set with EX 2)
       None
redis> GEOSEARCH farms:geo FROMLONLAT 3.983 6.583 BYRADIUS 60 km ASC WITHDIST
       23 farms
       [1, '70']
       [0, '70']
       call 1: cache MISS   42.79 ms  (9 months)
       call 2: cache HIT     0.56 ms  (9 months)
       used_memory_human = 2.73M, DBSIZE = 2193 keys
       aof_enabled = 1, appendfsync = everysec, maxmemory-policy = volatile-lru
```

Figure 5: Synchronous Replication (SyncRep) Blocking, Sharded Query Plan and Failover.

```
================================================================================================
R6  SYNC REPLICATION WITH THE REPLICA DOWN - consistency over availability
================================================================================================
  replica1 stopped.
  INSERT on primary still waiting after 5 s -> writes are BLOCKED
  pid | state | wait_event_type | wait_event | query
  14296 | active | IPC | SyncRep | INSERT INTO aqua.replication_heartbeat (note) VALUES ('durin
  Operator decision: drop to asynchronous replication to restore availability.
  -> the blocked INSERT committed 5.0 s after it started.
  replica1 restarted and caught up from the replication slot:
  application_name | state | sync_state | bytes_behind
  replica1 | streaming | async | 0
  row written during the outage now on replica: True

[saved to evidence/replication_demo_output.txt]
```

```
================================================================================================
S4  CROSS-SHARD QUERY - readings and average oxygen per farm (scatter-gather)
================================================================================================
  4a pushdown OFF (raw rows shipped to the coordinator):   1406.9 ms
  4b pushdown ON  (each shard runs its own GROUP BY)     :    374.9 ms
  4c same aggregate on the single-server table           :    509.5 ms

Plan with pushdown (note Async Foreign Scan on both shards and the Remote SQL):
Append
  ->  Async Foreign Scan
        Output: wqr_dist.farm_id, (count(*)), (round(avg(wqr_dist.dissolved_oxygen_mg1), 2))
        Relations: Aggregate on (aqua.wqr_dist_p0 wqr_dist)
        Remote SQL: SELECT farm_id, count(*), round(avg(dissolved_oxygen_mg1), 2) FROM public.readings GROUP BY 1
  ->  Async Foreign Scan
        Output: wqr_dist_1.farm_id, (count(*)), (round(avg(wqr_dist_1.dissolved_oxygen_mg1), 2))
        Relations: Aggregate on (aqua.wqr_dist_p1 wqr_dist_1)
        Remote SQL: SELECT farm_id, count(*), round(avg(dissolved_oxygen_mg1), 2) FROM public.readings GROUP BY 1
```

```
================================================================================================
F3  FAILOVER: promote replica1 to primary
================================================================================================
  pg_promote() returned True
  in_recovery=False
  new timeline=2
  write accepted on new primary, id=661
  RTO (crash -> promotion complete): 4707 ms;  crash -> first successful write: 6237 ms
  (manual, scripted promotion; real RTO also includes failure detection, e.g. Patroni's ~30 s leader TTL)
  RPO: row 633 ('last write before crash') present on the new primary -> True  => 0 rows lost
```

Figure 6: Cloud Architecture.

```
7]: Image("cloud_architecture.png")
```

